Consulta, tratamiento y análisis de datos meteorológicos

Reto de programación con Python

En este proyecto consultarás datos meteorológicos históricos de Jaén mediante la API de Open-Meteo. Aprenderás a obtener datos desde Internet, organizarlos con Python, analizarlos y comunicar tus resultados.

No necesitas conocer previamente `requests` ni `matplotlib`: las incorporarás cuando las necesites.

Al terminar podrás

- Explicar qué es una API REST y realizar una petición HTTP GET.
- Interpretar una respuesta JSON y trabajar con listas y diccionarios.
- Limpiar, filtrar y transformar datos con Python.
- Calcular estadísticas descriptivas y representar series temporales.
- Guardar resultados y explicar qué muestran.

Ruta del proyecto

Avanza por las etapas en orden. En cada una, ejecuta el código, observa los resultados y responde a las preguntas con tus propias palabras.

1. Explora la API y comprende los datos que devuelve.
2. Realiza una petición desde Python.
3. Explora listas y diccionarios.
4. Calcula estadísticas con listas.
5. Transforma y filtra datos con comprensiones.
6. Organiza las temperaturas junto con sus fechas.
7. Analiza los datos meteorológicos.
8. Estudia los cambios a lo largo del tiempo.
9. Representa los resultados con gráficos.
10. Controla errores y valores ausentes.
11. Guarda los datos en archivos.
12. Amplía el análisis a otras ciudades, periodos y variables.
13. Divide el programa en funciones y prueba su funcionamiento.
14. Desarrolla y presenta tu proyecto final.

1. Explora la API

`Antes de programar, observa la información que proporciona Open-Meteo. Abre la consulta de temperaturas históricas de Jaén y examina la respuesta.`

Histórico temperaturas en Jaén

`Responde:`

1. ¿Qué información proporciona el servicio?
2. ¿Qué significan `latitude` y `longitude`?
3. ¿Qué fechas se están consultando?
4. ¿Qué información solicitan `temperature_2m_mean`, `temperature_2m_max` y `temperature_2m_min`?
5. ¿En qué formato se presenta la respuesta?
6. ¿Qué diferencias observas entre esa respuesta y una lista de Python?
7. ¿Qué ocurriría si cambiásemos `end_date` para consultar hasta el 15 de enero?
8. ¿Qué ventaja tiene consultar una API frente a introducir los datos manualmente?

2. Realiza una petición desde Python

La biblioteca `requests` permite realizar peticiones HTTP. Instálala desde la terminal:

```
python -m pip install requests
```

Crea un archivo llamado `meteorologia.py` y escribe este programa:

```
import requests

url = "https://archive-api.open-meteo.com/v1/archive"
parametros = {
    "latitude": 37.7796,
    "longitude": -3.7849,
    "start_date": "2025-01-01",
    "end_date": "2025-01-31",
    "daily": "temperature_2m_mean,temperature_2m_max,temperature_2m_min",
    "timezone": "Europe/Madrid",
}

respuesta = requests.get(url, params=parametros, timeout=30)
respuesta.raise_for_status()
datos = respuesta.json()

print(datos)
```

Ejecuta el programa y observa el resultado. `requests.get()` envía la petición; `raise_for_status()` informa de errores HTTP y `respuesta.json()` convierte la respuesta JSON en objetos de Python. No tienes que importar `json`: viene incluido en Python.

Comprueba tu comprensión:

1. ¿Para qué sirve `import requests`?
2. ¿Qué tipo de objeto es `parametros`?
3. ¿Qué diferencia hay entre `url` y `parametros`?

4. ¿Qué hacen `timeout=30` y `raise_for_status()`?
5. ¿Qué diferencia hay entre `respuesta` y `datos`?
6. ¿Qué podría ocurrir si falla la conexión o el servidor devuelve un error?
7. ¿Por qué separamos los parámetros de la dirección URL?
8. ¿En qué se diferencia una petición GET de una petición POST?

3. Explora la estructura de los datos

Examina el diccionario `datos`. Puedes empezar mostrando sus claves:

```
print(datos.keys())
```

Usa las claves para localizar la información diaria. Guarda ese diccionario en `diarios` y responde:

1. ¿Qué valores tienen las claves `latitude` y `longitude`?
2. ¿Qué claves contiene `diarios`?
3. ¿Qué unidades se utilizan para la temperatura?
4. ¿Qué tipos de dato devuelven `type(datos['daily'])`, `type(datos['daily']['time'])` y `type(datos['daily']["temperature_2m_mean"])`?
5. ¿Cuántas fechas y temperaturas has obtenido?
6. Comprueba que las listas de fechas, temperaturas medias, máximas y mínimas tienen la misma longitud. ¿Por qué deben coincidir?
7. Muestra la primera fecha, la temperatura media del primer día y la máxima del último.
8. ¿Qué sucede si intentas acceder a una posición que no existe?
9. ¿Cómo comprobarías si una clave está presente antes de utilizarla?

4. Calcula estadísticas con las temperaturas

Guarda la lista de temperaturas medias en una variable:

```
temperaturas = datos['daily']["temperature_2m_mean"]
print(temperaturas)
```

Practica con esta lista. Calcula la suma, la media, el valor máximo y el mínimo. Después responde:

1. ¿Cuáles son las cinco primeras y las cinco últimas temperaturas?
2. ¿Cuántos valores contiene la lista?
3. ¿Cuál es la diferencia entre la temperatura más alta y la más baja?
4. ¿Cuántos días superaron los 15 °C? ¿Cuántos estuvieron por debajo de 10 °C?
5. Crea una lista con las temperaturas superiores a 15 °C y otra con temperaturas inferiores a 10 °C.
6. Comprueba si todas las temperaturas son superiores a 0 °C y si alguna supera los 20 °C.
7. Redondea los valores a un decimal y guarda el resultado en otra lista.
8. Ordena las temperaturas de menor a mayor sin alterar la lista original.

9. Calcula la diferencia entre cada temperatura y la media del periodo. Nos interesa los valores absolutos.

Recuerda: la temperatura media de un día no es lo mismo que su temperatura máxima.

5. Transforma y filtra datos

Utiliza comprensiones de listas para crear nuevas colecciones. La conversión de grados Celsius a Fahrenheit es:

$$F = \frac{9}{5}C + 32$$

Realiza estas actividades:

1. Convierte las temperaturas a Fahrenheit.
2. Crea listas con temperaturas en dos rangos, un rango con las temperaturas menor o igual a la media y otra como superior a la media, usa las temperaturas en ºC.
3. Clasifica cada temperatura como inferior, igual o superior a 15 °C.
4. Crea una lista con los índices de los días cuya temperatura media superó los 15 °C.
5. Construye tuplas con cada temperatura en Celsius y su equivalente en Fahrenheit.
6. Calcula la variación respecto al día anterior.
7. Define umbrales para clasificar los días como fríos, templados o cálidos y explica tu decisión.

6. Relaciona cada valor con su fecha

Las temperaturas están asociadas a fechas. Organiza los datos en una colección de registros que conserve, como mínimo, la fecha y las temperaturas media, máxima y mínima. Crea el siguiente diccionario, donde una entrada del mismo sería:

```
fecha: {
        'media': media,
        'maxima': maxima,
        'minima': minima
    }
```

Comprueba que puedes mostrar un registro completo, localizar el día con la media más alta y calcular la amplitud térmica diaria (máxima menos mínima).

7. Analiza el periodo

Calcula medidas descriptivas para las temperaturas medias, máximas y mínimas. Estas son:

- Media aritmética: es el valor que obtenemos al sumar todas las temperaturas y dividir el resultado entre el número de temperaturas.
- Mediana: es el valor que queda en el centro cuando ordenamos todas las temperaturas de menor a mayor (si tenemos un número impar de temperaturas, la mediana es el valor central, si tenemos un número par, la mediana es la media de los dos valores centrales)
- Valor máximo y mínimo: son, respectivamente, el mayor y el menor valor del conjunto.
- Rango: es la diferencia entre el valor máximo y el valor mínimo.

- Varianza: es una medida que indica cuánto se alejan, en conjunto, los valores respecto a la media, para calcularla, se determina la diferencia entre cada temperatura y la media, se elevan esas diferencias al cuadrado y se calcula su promedio
- Desviación típica: es la raíz cuadrada de la varianza. Nos permite medir la dispersión de los valores.

Escribe conclusiones explicando qué significa cada resultado. No presentes únicamente números: indica qué muestran sobre el periodo estudiado.

8. Estudia la evolución temporal

1. Muestra las temperaturas medias junto con sus fechas.
2. Filtra la primera semana y la segunda quincena.
3. Calcula las variaciones entre días consecutivos e identifica el mayor aumento y descenso.
4. Calcula una media móvil de tres días y compárala con los valores originales.
5. Divide el periodo en semanas y calcula la media de cada una.

9. Representa los resultados

Instala `matplotlib` si todavía no está disponible:

```
python -m pip install matplotlib
```

Crea gráficos para mostrar la evolución de las temperaturas medias, máximas y mínimas; compara también las semanas y representa las amplitudes térmicas. Añade títulos, etiquetas y leyendas. Guarda los gráficos en formato PNG.

Se crearan los siguientes gráficos:

- Gráfico de líneas: evolución de las temperaturas: el gráfico de líneas es una buena primera opción cuando queremos observar cómo cambian las temperaturas a lo largo del tiempo. Indica título y leyendas. Cada línea de distinto color.
- Gráfico de barras: comparar temperaturas. Un gráfico de barras permite comparar valores concretos entre categorías, podemos comparar los valores semanales, tanto de la temperatura media, máxima y mínima, usando los datos del apartado 8.
- Histograma: distribución de las temperaturas. El histograma permite estudiar cómo se distribuyen los valores. En lugar de centrarnos en qué día se registró una temperatura, observamos qué intervalos de temperatura son más frecuentes.
    - Representar mediante un histograma la distribución de las temperaturas medias. Identificar el intervalo de temperaturas que contiene más días.
    - Determinar si las temperaturas están concentradas en torno a un valor o están muy dispersas.
    - Repetir el histograma para las temperaturas máximas y mínimas.
    - Comparar los tres histogramas y explicar las diferencias.
- Diagrama de dispersión: relación entre temperaturas máximas y mínimas. Intenta obtener conclusiones de la posible relación entre ambas variables. ¿Se podría predecir la temperatura máxima de un día conocida la temperatura mínima?

Al terminar, explica qué gráfico te ayuda más a estudiar la evolución temporal y redacta tres conclusiones basadas en tus representaciones.

10. Controla errores y datos ausentes

Un programa debe responder de forma comprensible cuando falla la conexión, la API tarda demasiado o el servidor devuelve un error. Practica la gestión de `Timeout`, `ConnectionError`, `HTTPError` y otros errores de petición.

Comprueba también que la respuesta contiene la clave `daily`, que las listas tienen la misma longitud y que no hay valores ausentes antes de calcular estadísticas. Explica por qué sustituir automáticamente un valor ausente por cero puede llevar a conclusiones incorrectas.

11. Guarda los resultados

Guarda los registros en JSON y CSV. Después, vuelve a leerlos y comprueba que coinciden con los datos originales. Añade una columna con la amplitud térmica y crea otro CSV que contenga solo la fecha y la temperatura media.

Compara las ventajas de JSON y CSV y organiza los archivos con nombres que indiquen el periodo analizado.

12. Amplía la investigación

Elige una o varias ampliaciones:

- Compara Jaén, Madrid y Sevilla usando sus coordenadas. Organiza los resultados por ciudad y represéntalos en un gráfico.
- Compara enero de 2024 con enero de 2025. Calcula las medias, compara máximas y mínimas y explica por qué dos meses no bastan para establecer una tendencia climática.
- Añade la precipitación diaria (`precipitation_sum`, en milímetros). Calcula el total, la media diaria y el porcentaje de días con precipitación; identifica el día más lluvioso y crea un gráfico.

13. Organiza y prueba tu programa

Divide el programa en funciones. Puedes crear funciones para obtener los datos, calcular la media y construir los registros. Cada función debe tener una responsabilidad clara, recibir los datos que necesita y devolver un resultado.

Prueba las funciones con datos conocidos, incluidos casos límite como una lista vacía, un único valor y temperaturas negativas. Las pruebas te permiten comprobar el análisis sin depender de la conexión con la API.

Proyecto final: informe meteorológico

Desarrolla una pequeña aplicación que permita elegir una ciudad y un periodo, obtener datos históricos, calcular estadísticas, mostrar un resumen, guardar los resultados y generar gráficos. Organiza el código en funciones y controla los errores habituales.

Entrega:

- El código fuente.
- Los datos obtenidos en CSV o JSON.

- Al menos dos gráficos.
- Una breve explicación de los resultados y sus limitaciones.

En tu informe responde: ¿cuál fue la temperatura media del periodo?, ¿qué días registraron las máximas y mínimas más destacadas?, ¿cuál fue la amplitud térmica diaria media?, ¿qué diferencias hubo entre semanas?, ¿qué muestran los gráficos y qué limitaciones tiene el análisis?